

Lecture 33 — Trusted Computing

Jeff Zarnett

2026-08-23

Whose PC is it Anyway?

In the discussion of kernel modules, we learned that online multiplayer games use kernel-level anti-cheat mechanisms. Even that is sometimes not sufficient from the view of game developers. In 2025, for example, Activision/Blizzard announced that they would require TPM 2.0 and Secure Boot to play Call of Duty. These two things are hardware-based functionality that are ostensibly about security, though as we will see, things are slightly more complex than that, because these security features open questions about whom we are trying to secure the computer from, and why.

The focus on hardware-level protection of what is allowed to run on the computer started around the year 2000, with the publication of the version 1.0 specification of Trusted Computing. It is *not at all* a coincidence that this conversation started at a time where the hot topic amongst the megacorporations was “stopping music piracy”.

History lesson on this: in June of 1999, the application Napster launched, which was a peer-to-peer music sharing service that let people download copies of mp3 music files shared by other users, without paying. This made a number of big media conglomerates 10 out of 10 angry, because they argued that every downloaded (pirated) copy was a lost sale and therefore they were losing millions upon millions of dollars. I personally don't think this is true, but there were a lot of lawsuits and things got ugly. The big players were looking to stop this through something called Digital Rights Management (DRM): the ability to stop you from doing things they do not want you to do with the media that you purchased.

Digital Rights Management is a little euphemistic, because it is really digital restrictions management. These sorts of things only work if the software understands what the restrictions are supposed to be and enforces them. So perhaps the video player permits you to view the video but not to save it, or the reader software permits you to read an e-book but not take a screenshot. You can very likely think of some workarounds for these restrictions immediately, if you control the computer and its software.

For example, if the media player has the download function but it's (sometimes) disabled, you can just... edit the application binary so the if-else condition that determines if this button should be enabled always says it is enabled. If the media is encrypted, you can find the encryption key in the process memory and use that to decrypt the media. If there's some network call to check for authorization, you can redirect that call to a local server you control that always says yes. If the operating system forbids copying the file somehow, you can just override that with a kernel module that lets you do it. “Do whatever you want with your purchased content with this one weird trick! Corporations hate him!”

Security, as discussed previously, is always an arms race. The key factor in all of the examples given above is that the person who purchased and administers the computer has a lot of ability to modify application software and even the OS. Any of the examples just given that would defeat restrictions measures is not trivial; most people probably do not have the knowledge or skills to do it. Still, if even one person creates an unencrypted copy of the movie and shares it, it can propagate and be watched by anybody. Which, as we know, the movie studios desperately want to stop. So the prevention of circumvention of DRM has to move lower down, into hardware, because that's the only way to break that condition where the administrator of the computer has enough control to bypass restrictions.

To quote a simple explainer: “TC provides a computing platform on which you can't tamper with the application software, and where these applications can communicate securely with their authors and with each other. The original motivation was digital rights management (DRM): Disney will be able to sell you DVDs that will decrypt and run on a TC platform, but which you won't be able to copy. The music industry will be able to sell you music

downloads that you won't be able to swap. They will be able to sell you CDs that you'll only be able to play three times, or only on your birthday." [And03].

It's not in the article, because that is from 2003 and a great many things have gotten worse now, but the current state of DRM is that it allows the corporation to alter the deal at any time. In 2026, for example, Sony deleted movies (including *Terminator 2: Judgement Day*) that customers paid for, removing it from their accounts with no refund or compensation [Wal26]. Yes, yes, they say things like "you are purchasing a *license* to watch the movie under limited circumstances" and there are more terms buried in the legalese about their right to do this when their licensing deal with the movie studio ends, but whether or not it is technically legal does not diminish the danger here: they *can* do this and the consumer has no way to stop it.

From what we already discussed, we can see why a hardware-level solution is necessary if the goal is to meaningfully enforce these restrictions. If I, as the user, control the kernel – up to and including live-patching it – I can bypass restrictions. If the computer will refuse to boot a kernel that is not "approved" by the authorities (and such an approved kernel would, naturally, forbid any sort of live-patching) it is much more difficult to work around the restrictions that the content companies (or others) want to impose¹.

There are further concerns outlined in [And03] that this could go beyond just preventing people from watching media or copying software, and that it can be used to delete content that the government, or other powerful actors, don't want you to have. The standard dystopian example of this: imagine a journalist has published an exposé on the corruption and criminality of the dictator of a country, and said dictator simply has the ability to order that all copies of that document are deleted from everyone's computer.

At least at present, if verification fails, the computer is not useless: it's just in an "untrusted" state, and that restricts access to certain things. This means that you can still boot your operating system, run some applications, and do lots of things. But not play the desired game or consume restricted media.

It is, however, not unthinkable that the future state is that it will be difficult to run the computer in an untrusted state at all². Yes, it is likely that Linux and other open-source options will exist, but as life becomes increasingly conducted online, it gets harder and harder over time to use a "free" computer if you cannot use it do the things needed in everyday life: edit Word/Google documents, use online banking, pay taxes, order online from popular merchants, etc. Will you need two computers? Oh probably.

Less-Corporate-Overlord Reasons. While I do think the primary motivation for the trusted computing platform really is to take the power out of your hands as the owner of the machine and let the megacorporations decide what you are "allowed" to do, there are practical, security-focused reasons for trusted computing and secure boot.

Gaming provides a legitimate example. If you want to be sure that the other players are not cheating, the kernel-level anti-cheat module can be backstopped by the trusted computing platform: if the kernel (and its extensions) are not all "approved", that means the anti-cheat module cannot guarantee the integrity of the system (because, potentially, some cheating kernel extension is loaded) and will therefore decline the request to start the game.

Another security benefit is defeating low-level compromises of the system that the OS itself may struggle to detect. If malware is installed in the system firmware or boot loader, the malware can survive if the OS is reinstalled. The OS cannot find it, because the OS is running itself on a compromised foundation. The supposed solution to that is called Secure Boot. The Arch Linux wiki defines this as a security feature in the UEFI standard to verify these low-level foundational components to be sure they are valid.

Unsurprisingly, these mechanisms are not perfect either. There are numerous examples of bypassing secure boot out there.

¹With that said, people almost always FIND a way.

²They're getting pretty close with phones and tablets.

References

- [And03] Ross Anderson. 'trusted computing' frequently asked questions, 2003. Online; accessed 16-August-2026. URL: <https://www.cl.cam.ac.uk/archive/rja14/tcpa-faq.html>.
- [Wal26] John Walker. Playstation is deleting 551 movies from customers' accounts, reminding us nothing digital is ever truly ours, 2026. Online; accessed 17-August-2026. URL: <https://kotaku.com/playstation-store-movies-digital-studio-canal-terminator-2000711013>.